CachePilot — Master Specification

Semantic Caching & Cost-Aware Model Routing for RAG

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Product overview

RAG applications can spend too much on repeated questions and route trivial requests to expensive models. They also need evidence that savings do not silently reduce answer quality.

Objectives and non-goals

- Build an observable RAG gateway that combines embedding-similarity caching, policy-driven model routing, grounded answer generation, per-request cost and latency accounting, and an evaluation harness.
- Deliver a runnable local development environment with deterministic setup instructions and sample data.
- Provide API contracts, automated tests, security controls, operational telemetry, and reproducible evaluation evidence.
- Keep external services behind adapters so providers can be replaced without rewriting business logic.
- Non-goal: claiming production certification, guaranteed savings/accuracy, or enterprise compliance without measured evidence.

Architecture and technology

- Frontend: React + TypeScript + Vite + Tailwind CSS; typed API client; accessible loading, empty, error, and permission-denied states.
- Backend: Python + FastAPI + Pydantic; layered modules for API, services, repositories, domain models, adapters, and configuration.
- Persistence: PostgreSQL with migrations; pgvector for embeddings; Redis where caching/queues are needed.
- LLM/embedding providers: interfaces plus at least one configurable implementation and a deterministic mock provider for tests.
- Deployment: Docker Compose for local services; health checks; .env.example containing names and safe placeholders only.
- Testing: Pytest for unit/integration tests; frontend component tests; API contract tests; end-to-end smoke tests.

Functional requirements

- Request API: Accept query, tenant/user context, conversation context when enabled, requested response mode, and correlation ID. Validate payloads and apply rate limits.
- Policy and cache safety: Normalize query text; construct a cache key that includes tenant, authorization scope, embedding/model version, corpus version, prompt version, and relevant generation settings. Never share cached answers across incompatible security scopes.
- Semantic cache: Create query embeddings; retrieve candidate cached questions; use a configurable similarity threshold and optional exact-match fallback. Return a cache hit only when scope and freshness checks pass. Support TTL, invalidation, and cache bypass.

- Cost-aware router: Classify request complexity using configurable rules and/or a lightweight model. Select small/large model based on complexity, context size, required reasoning, confidence, latency target, and budget policy. Provide deterministic fallback when a provider fails.
- RAG pipeline: Retrieve relevant chunks, assemble bounded context, generate cited answers, and return source references. Keep retrieval and generation separately measurable.
- Usage ledger and dashboard: Record model/provider, token usage when available, estimated cost, cache status, retrieval/generation latency, total latency, route reason, quality signals, and errors. Label estimates separately from provider-reported billing.
- Evaluation and operations: Run a fixed benchmark of representative questions, compare cache/routing variants against a baseline, track quality regressions, and support configuration changes without code edits.

Data model

tenants, users, documents, chunks, cache_entries, requests, model_usage, routing_policies, evaluation_runs, evaluation_results. Include tenant_id and version fields where applicable; add indexes for tenant/scope, timestamps, cache expiry, and request correlation IDs.

API contract baseline

- GET /health/live — process liveness; GET /health/ready — dependency readiness.
- POST /api/v1/documents — ingest a document or submit an ingestion job.
- GET /api/v1/documents — list only documents visible to the authenticated user/scope.
- POST /api/v1/query — run retrieval and answer generation; return answer, citations/evidence, trace ID, and safe diagnostic metadata.
- GET /api/v1/metrics/summary — return dashboard aggregates for authorized users.
- GET /api/v1/requests/{request_id} — fetch permitted request diagnostics without exposing secrets or unauthorized content.
- Use Pydantic request/response schemas, pagination, consistent error envelopes, request IDs, and explicit API versioning.

Security, privacy, and reliability

- Validate all inputs; enforce authentication and authorization on every protected route; apply rate limits and request-size limits.
- Use least privilege for database/service accounts; parameterized queries; safe CORS; secure headers; no secrets in source control or logs.
- Define retention and deletion behavior for documents, embeddings, cache entries, graph facts, and telemetry.
- Treat documents and retrieved text as untrusted data; defend against prompt injection and instructions embedded in source material.
- Use timeouts, bounded retries with backoff, circuit breakers or provider fallback where appropriate, and graceful degradation.
- Log identifiers and operational metadata rather than raw sensitive content by default.

Observability and evaluation

- Emit structured logs with correlation/request ID and component-level timings.
- Measure request volume, errors, retrieval latency, generation latency, total p50/p95 latency, token usage when available, and estimated cost.
- Maintain a versioned evaluation dataset with expected answer/evidence and documented scoring rules.

- Report quality together with performance/cost; do not present latency or cost improvements without a comparable baseline.
- Store model, prompt, embedding, corpus/index, and policy versions with each evaluation/request trace.

Acceptance criteria

- Exact and semantic cache hits return valid responses only within the same cache/security scope.
- Cache invalidation and TTL work; changing corpus, prompt, embedding model, or relevant generation settings prevents stale reuse.
- Router selects models according to documented rules and records a human-readable routing reason.
- Every request has a correlation ID and records latency, route, cache status, token usage when available, estimated cost, and error outcome.
- Dashboard displays request volume, cache hit rate, estimated cost, p50/p95 latency, model distribution, failures, and quality evaluation results.
- Benchmark compares baseline and optimized configurations with the same dataset and reports quality alongside cost and latency.
- Automated tests cover cache isolation, routing rules, provider failures, invalidation, and API validation.

Risks and mitigations

- Similarity thresholds can return a semantically related but incorrect answer; require evaluation and conservative defaults.
- Cached answers can become stale; version keys, TTL, and explicit invalidation are mandatory.
- Cost figures may be estimates if providers do not expose authoritative billing; show assumptions and units.
- Routing can lower quality; retain a high-quality fallback and evaluate by question category.

Repository structure

frontend/
backend/app/{api,core,domain,services,repositories,adapters,schemas}/
backend/tests/{unit,integration,security,evaluation}/
migrations/
infra/docker-compose.yml
scripts/
data/sample/
docs/{architecture,api,threat-model,evaluation,runbook}/
.env.example
README.md

Definition of done

- A new developer can follow README instructions to start the app from a clean checkout.
- Database migrations and seed data run reproducibly.
- Unit, integration, security-critical, and frontend build tests pass.
- No secrets are committed; .env.example contains only placeholders.
- API docs, architecture notes, threat model, test results, known limitations, and demo instructions are present.
- The final report distinguishes implemented features from planned work and measured results from estimates.

Product-specific design decisions

- Cache key must include tenant/security scope, normalized query, embedding model/version, corpus/index version, prompt version, and generation parameters that affect answer validity.
- Semantic similarity is a candidate retrieval mechanism, not authorization. Always validate scope and freshness separately.
- Use conservative default threshold and provide exact-match, semantic-hit, miss, bypass, and invalidation paths.
- Routing policy is configurable and records a reason code. Model selection must not depend on hidden assumptions about provider pricing.
- Cost estimates use versioned provider price configuration and token counts; mark estimates as estimates unless confirmed billing data is available.